



Année universitaire 2025-2026

Rapport de projet 3D game programming

Présenté par : MATTHIEU Rémi & SANCHEZ Louise

Sous la direction de M. Buffa

Table des matières

1	Introduction	2
2	Développement et fonctionnalités	2
3	IA du blackjack : Monte Carlo et approche pratique	3
4	IA du poker : IS-MCTS et défis de l'information incomplète	3
5	Machine à sous et utilité dans le prototype	4
6	Conclusion et perspectives	4

1 Introduction

Dans le cadre de la matière 3D game programming dispensée au cours du 2eme semestre du Master 1 Informatique à l'université Nice Côte d'Azur, nous avons été amenés à réaliser un jeu vidéo en 3D. Le jeu vidéo réalisé dans cette matière doit utiliser la librairie BabylonJS et doit être présenté au concours Games On Web. Games On Web est un concours de développement de jeux vidéo en équipe entre les élèves de plusieurs universités. Ce rapport retrace les différentes étapes de développement du projet, les choix techniques effectués, les difficultés rencontrées et les perspectives d'amélioration pour le futur.

2 Développement et fonctionnalités

Le projet repose sur un environnement de casino en 3D dans lequel le joueur peut se déplacer librement et interagir avec différentes stations de jeu. Plusieurs modules ont été développés afin de gérer le déplacement du personnage, les interactions avec les tables ainsi que les différentes logiques de jeu présentes dans le prototype.

Au départ j'ai cherché à utiliser des modèles glTF exportés depuis Blender et à retargeter des animations, mais j'ai rapidement rencontré un problème de centrage : certaines animations provoquaient un déplacement du personnage plutôt qu'une marche sur place. Après plusieurs tentatives pour recentrer le squelette sans succès, j'ai choisi une solution pragmatique et efficace pour le temps du projet : utiliser un modèle Mixamo déjà animé et adapter son placement dans la scène. Cette décision a permis de consacrer le temps disponible aux mécaniques et à l'IA plutôt qu'à des détails superficiels.

Le déplacement du personnage a été développé avec un contrôleur personnalisé. J'ai essayé une bibliothèque externe au début, mais elle posait plusieurs problèmes (inversions de touches, diagonales mal gérées, comportements difficiles à corriger). J'ai donc décidé, à la place, d'implémenter un contrôleur basé sur une capsule physique invisible inspirée d'exemples officiels de Babylon.

Le projet utilise le moteur physique Havok à travers le plugin physique de BabylonJS. Havok est utilisé pour gérer les collisions, la gravité et les différents corps physiques présents dans la scène. La capsule physique du joueur repose sur ce système et permet de gérer les déplacements et les interactions avec le décor du casino de manière stable.

La capsule gère la collision et la physique, tandis que le modèle visible reste attaché à un nœud séparé pour permettre une rotation fluide. Les entrées clavier sont lues par `src/core/InputManager.js`, transformées en un vecteur relatif à la caméra via `src/camera/CameraControl` puis appliquées à la capsule.

Les animations sont chargées depuis le fichier du personnage et sélectionnées par nom.

Pour l'environnement, le chargement du casino se fait via des modèles exportés depuis Blender. Les collisions sont gérées par des boîtes créées manuellement dans Blender et nommées avec le préfixe `collider_`. À l'import, le code crée pour chacune une boîte physique statique invisible, ce qui simplifie le placement des volumes de collision sans ajout de logique complexe. Un sol de sécurité est également présent sous le casino pour éviter les chutes imprévues. Un bug rencontré pendant le développement était une symétrie involontaire causée par une modification du nœud racine après l'import glTF ; la correction consistait à ne pas

altérer la compensation automatique du chargeur glTF et à laisser le nœud racine intact.

Les interactions (s'asseoir, quitter, démarrer une partie) sont gérées par `src/interaction/Interaction`. En approchant une station, un point d'interaction devient actif et déclenche une transition de caméra et de contrôles. Cette mécanique rend le passage entre exploration et jeu plus naturel.

3 IA du blackjack : Monte Carlo et approche pratique

L'IA du croupier pour le blackjack est simple. Le croupier doit choisir entre tirer ou rester, et l'espace d'action est réduit, ce qui rend une approche par simulation statistique particulièrement adaptée. Dans `src/games/black-jack/ai/DealerMonteCarloAI.js`, l'algorithme évalue les deux options en lançant pour chacune un grand nombre de simulations. Chaque simulation clone l'état du paquet et des mains, joue la suite selon la règle « tirer jusqu'à 17 » et compare l'issue avec la main du joueur.

Il s'agit d'une forme de contrôle Monte Carlo : au lieu de rechercher exhaustivement un arbre d'actions, on estime la valeur d'une action par des simulations aléatoires répétées. Cette méthode est particulièrement efficace pour le blackjack car l'horizon de décision est court et l'incertitude porte essentiellement sur l'ordre des cartes restantes. Le coût de calcul est maîtrisé en ajustant le nombre d'itérations, et le résultat est une estimation statistique de la probabilité de gain pour chaque action.

Le lien avec des méthodes plus générales de Monte Carlo Control vient du principe commun : estimer des valeurs d'actions par échantillonnage et choisir celle ayant la meilleure espérance. Dans un système de Monte Carlo Control, on améliore progressivement une politique basée sur des estimations de valeur.

4 IA du poker : IS-MCTS et défis de l'information incomplète

Le poker a une complexité plus importante que le blackjack. Les cartes adverses sont cachées, le nombre d'actions possibles inclut des montants variés, et la stratégie optimale dépend fortement des comportements adverses. Pour traiter ces difficultés, j'ai mis en place une version simplifiée d'IS-MCTS (Information Set Monte Carlo Tree Search) dans `src/games/poker/ia/ISMCTS`.

L'approche consiste à générer, pour chaque itération, une « déterminisation » du monde : on échantillonne des mains adverses plausibles en retirant du paquet les cartes publiques et en tirant au hasard selon des distributions de mains. Ces mains sont fournies et ajustées par `src/games/poker/ia/OpponentRanges.js` et représentent des hypothèses sur ce que les adversaires peuvent tenir. Sur ce monde déterminisé, on exécute un MCTS classique : sélection via UCB, expansion, simulation rapide (rollout) et rétro-propagation de la récompense.

Les rollouts sont optimisés pour être à la fois rapides et crédibles. Plutôt que d'utiliser des jeux purement aléatoires, j'ai introduit des heuristiques simples qui reflètent des comportements humains plausibles, comme une tendance à se coucher face à des relances importantes ou à suivre pour de petites mises. La valeur retournée par un rollout est l'espérance de gain

en jetons pour l'agent, ce qui permet de comparer des actions de natures différentes (call, fold, raise partiel, raise agressif).

La sélection par UCB (Upper Confidence Bound) est employée pour équilibrer l'exploration et l'exploitation dans l'arbre de recherche. UCB favorise les actions qui ont montré de bonnes récompenses tout en encourageant l'examen d'actions peu explorées. Dans un jeu à information incomplète, ce mécanisme s'applique à chaque détermination et, en moyenne, converge vers des actions robustes si les mondes échantillonnés reflètent correctement la distribution réelle des mains adverses.

Plusieurs défis pratiques sont apparus. Le premier est la contrainte de temps : l'ISMCTS est coûteux et il est nécessaire de limiter le nombre d'itérations pour garder des temps de décision acceptables en jeu. Le second est la sensibilité aux ranges et aux heuristiques de rollout : si les mondes générés sont biaisés, l'IA peut prendre des décisions facilement exploitables. Pour réduire ce risque, j'ai combiné l'ISMCTS avec des règles de sécurité (par exemple, éviter des mises extrêmes dans des situations fragiles) et des ranges conservatrices par défaut.

5 Machine à sous et utilité dans le prototype

La machine à sous implémentée dans le prototype est volontairement simple. Elle a été générée à 100% avec l'ia et sert principalement d'outil de test et de moyen pour le joueur de récupérer des jetons si sa réserve est faible. Je m'en sert par exemple pour générer une réserve de jetons afin de tester de longues parties de poker avec plusieurs joueurs. À terme, ce système pourra être enrichi par des mécaniques de risque et de progression.

6 Conclusion et perspectives

Ce projet a permis de développer un prototype de casino en 3D intégrant plusieurs jeux, un système de déplacement libre, des interactions avec l'environnement et différentes logiques d'IA pour le blackjack et le poker.

Une partie importante du projet repose sur l'utilisation du moteur physique Havok via le plugin physique de BabylonJS. Havok est utilisé pour gérer les collisions, les corps physiques et les déplacements du joueur dans l'environnement. Le contrôleur du personnage repose notamment sur une capsule physique invisible, ce qui permet de simplifier la gestion des collisions avec le décor du casino tout en conservant des déplacements stables.

Les IA mises en place utilisent principalement des approches par simulation. Le blackjack repose sur des simulations Monte Carlo simples tandis que le poker utilise une version simplifiée d'IS-MCTS adaptée aux jeux à information incomplète.

Plusieurs améliorations restent possibles, notamment sur les performances des simulations, l'équilibrage des comportements adverses et l'ajout de nouveaux contenus dans le casino.